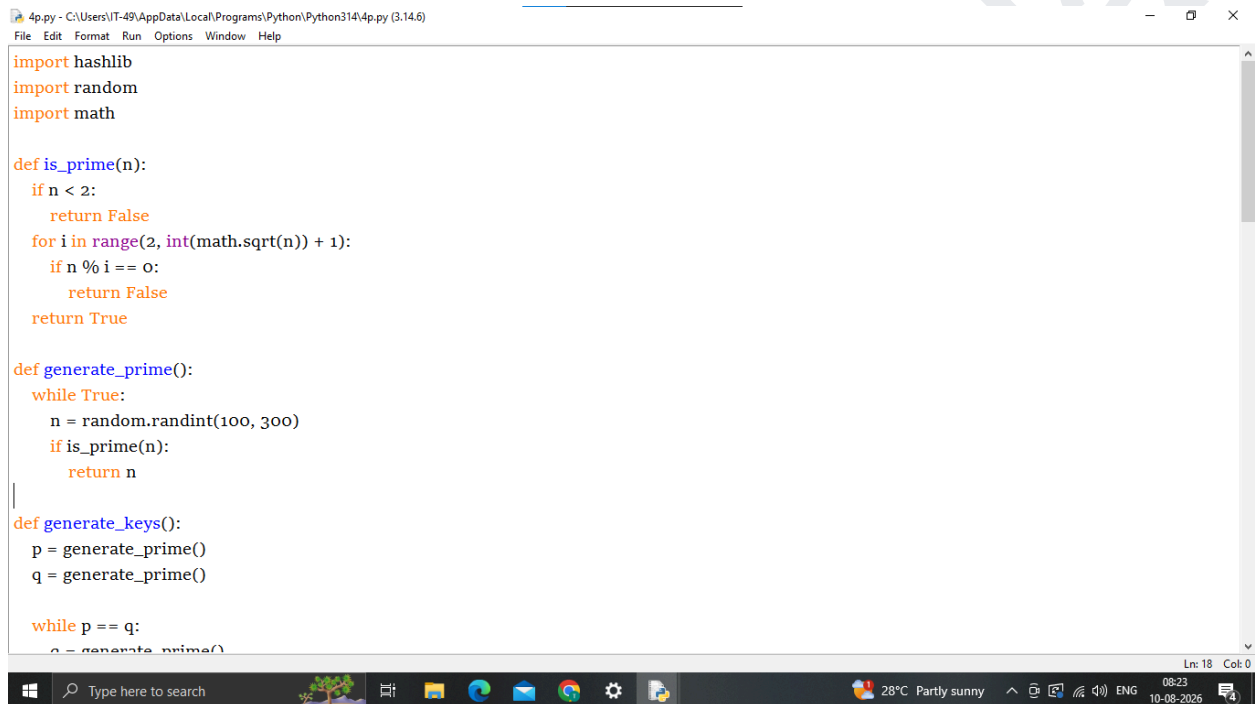


SHETH L.U.J AND SIR M.V. COLLEGE
SUBJECT NAME: Cyber & Information Security
Practical 4 : Digital Signature

Alm : Implement digital signature algorithms such as RSA-based signatures, and verify the integrity and authenticity of digitally signed messages.

Code:



```
4p.py - C:\Users\IT-49\AppData\Local\Programs\Python\Python314\4p.py (3.14.6)
File Edit Format Run Options Window Help

import hashlib
import random
import math

def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(math.sqrt(n)) + 1):
        if n % i == 0:
            return False
    return True

def generate_prime():
    while True:
        n = random.randint(100, 300)
        if is_prime(n):
            return n

def generate_keys():
    p = generate_prime()
    q = generate_prime()

    while p == q:
        q = generate_prime()
```

The screenshot shows a Python IDE window titled '4p.py - C:\Users\IT-49\AppData\Local\Programs\Python\Python314\4p.py (3.14.6)'. The code implements functions for RSA-based digital signature algorithms. It includes imports for 'hashlib', 'random', and 'math'. The 'is_prime' function checks if a number is prime. The 'generate_prime' function generates a random prime number. The 'generate_keys' function generates two distinct prime numbers, p and q. The code is written in Python 3.14.6. The Windows taskbar at the bottom shows the date as 10-08-2026 and the time as 08:23.

Name : adarsh shukla
Roll no : T117

SHETH L.U.J AND SIR M.V. COLLEGE
SUBJECT NAME: Cyber & Information Security
Practical 4 : Digital Signature

```
4p.py - C:\Users\UT-49\AppData\Local\Programs\Python\Python314\4p.py (3.14.6)
File Edit Format Run Options Window Help

q = generate_prime()

while p == q:
    q = generate_prime()

n = p * q
phi = (p - 1) * (q - 1)

e = 65537

if math.gcd(e, phi) != 1:
    e = 3
    while math.gcd(e, phi) != 1:
        e += 2

d = pow(e, -1, phi)

return (e, n), (d, n)

def sign_message(message, private_key):
    d, n = private_key

    hash_value = hashlib.sha256(message.encode()).hexdigest()
    hash_number = int(hash_value, 16)
```

```
4p.py - C:\Users\UT-49\AppData\Local\Programs\Python\Python314\4p.py (3.14.6)
File Edit Format Run Options Window Help

d = pow(e, -1, phi)

return (e, n), (d, n)

def sign_message(message, private_key):
    d, n = private_key

    hash_value = hashlib.sha256(message.encode()).hexdigest()
    hash_number = int(hash_value, 16)

    hash_number = hash_number % n

    signature = pow(hash_number, d, n)

    return signature

def verify_signature(message, signature, public_key):
    e, n = public_key

    hash_value = hashlib.sha256(message.encode()).hexdigest()
    hash_number = int(hash_value, 16)

    hash_number = hash_number % n
```

Name : adarsh shukla
Roll no : T117

SHETH L.U.J AND SIR M.V. COLLEGE
SUBJECT NAME: Cyber & Information Security
Practical 4 : Digital Signature

```
4p.py - C:\Users\UT-49\AppData\Local\Programs\Python\Python314\4p.py (3.14.6)
File Edit Format Run Options Window Help

def verify_signature(message, signature, public_key):
    e, n = public_key

    hash_value = hashlib.sha256(message.encode()).hexdigest()
    hash_number = int(hash_value, 16)

    hash_number = hash_number % n

    recovered_hash = pow(signature, e, n)

    return recovered_hash == hash_number

print("==== RSA DIGITAL SIGNATURE =====")

public_key, private_key = generate_keys()

print("\nPublic Key:", public_key)
print("Private Key:", private_key)

message = input("\nEnter message: ")

signature = sign_message(message, private_key)
```

```
4p.py - C:\Users\UT-49\AppData\Local\Programs\Python\Python314\4p.py (3.14.6)
File Edit Format Run Options Window Help

print("\nPublic Key:", public_key)
print("Private Key:", private_key)

message = input("\nEnter message: ")

signature = sign_message(message, private_key)

print("\nDigital Signature:", signature)

if verify_signature(message, signature, public_key):
    print("\nSignature Verification: SUCCESS")
    print("Message is authentic and unchanged.")
else:
    print("\nSignature Verification: FAILED")

modified_message = input("\nEnter message again to verify: ")

if verify_signature(modified_message, signature, public_key):
    print("Modified Message Verification: SUCCESS")
else:
    print("Modified Message Verification: FAILED")
    print("Message was changed.")
```

Output ;

Name : adarsh shukla
Roll no : T117

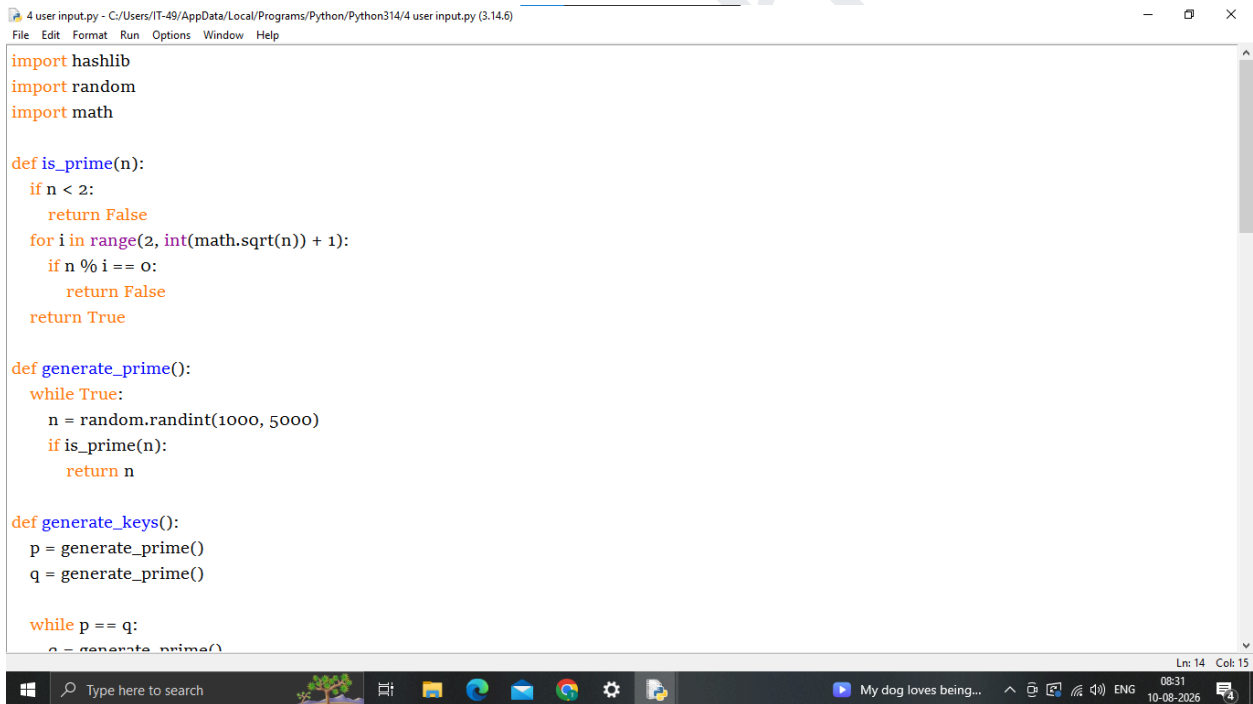
SHETH L.U.J AND SIR M.V. COLLEGE
SUBJECT NAME: Cyber & Information Security
Practical 4 : Digital Signature

```
>>>
==== RESTART: C:/Users/IT-49/AppData/Local/Programs/Python/Python314/4gui.py ====

===== RESTART: C:\Users\IT-49\AppData\Local\Programs\Python\Python314\4p.py =====
===== RSA DIGITAL SIGNATURE =====

Public Key: (65537, 48689)
Private Key: (37313, 48689)
```

#2 Implement digital signature algorithms such as RSA-based signatures, and verify the integrity and authenticity of digitally signed messages. User input .

A screenshot of a Python IDE window titled '4 user input.py - C:/Users/IT-49/AppData/Local/Programs/Python/Python314/4 user input.py (3.14.6)'. The window contains Python code for RSA key generation. The code imports hashlib, random, and math. It defines a function is_prime(n) to check for primality, a function generate_prime() to generate a prime number, and a function generate_keys() to generate a public and private key pair. The generate_keys function uses generate_prime to find two distinct primes p and q, and then enters a while loop to generate a private key d such that d * e ≡ 1 mod ((p-1)*(q-1)).

```
import hashlib
import random
import math

def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(math.sqrt(n)) + 1):
        if n % i == 0:
            return False
    return True

def generate_prime():
    while True:
        n = random.randint(1000, 5000)
        if is_prime(n):
            return n

def generate_keys():
    p = generate_prime()
    q = generate_prime()

    while p == q:
        q = generate_prime()
```

Name : adarsh shukla
Roll no : T117

SHETH L.U.J AND SIR M.V. COLLEGE
SUBJECT NAME: Cyber & Information Security
Practical 4 : Digital Signature

```
4 user input.py - C:/Users/IT-49/AppData/Local/Programs/Python/Python314/4 user input.py (3,14,6)
File Edit Format Run Options Window Help

q = generate_prime()

while p == q:
    q = generate_prime()

n = p * q
phi = (p - 1) * (q - 1)

e = 65537
if math.gcd(e, phi) != 1:
    e = 3
    while math.gcd(e, phi) != 1:
        e += 2

d = pow(e, -1, phi)

return (e, n), (d, n)

def sign_message(message, private_key):
    d, n = private_key
    hash_value = hashlib.sha256(message.encode()).hexdigest()
    hash_number = int(hash_value, 16) % n
    signature = pow(hash_number, d, n)
    return signature

Ln: 14 Col: 15
```

```
4 user input.py - C:/Users/IT-49/AppData/Local/Programs/Python/Python314/4 user input.py (3,14,6)
File Edit Format Run Options Window Help

return (e, n), (d, n)

def sign_message(message, private_key):
    d, n = private_key
    hash_value = hashlib.sha256(message.encode()).hexdigest()
    hash_number = int(hash_value, 16) % n
    signature = pow(hash_number, d, n)
    return signature

def verify_signature(message, signature, public_key):
    e, n = public_key
    hash_value = hashlib.sha256(message.encode()).hexdigest()
    hash_number = int(hash_value, 16) % n
    recovered_hash = pow(signature, e, n)
    return recovered_hash == hash_number

print("==== RSA DIGITAL SIGNATURE =====\n")

public_key, private_key = generate_keys()

print("Public Key:")
print(public_key)

Ln: 14 Col: 15
```

Name : adarsh shukla
Roll no : T117

SHETH L.U.J AND SIR M.V. COLLEGE
SUBJECT NAME: Cyber & Information Security
Practical 4 : Digital Signature

```
print("==== RSA DIGITAL SIGNATURE ==== \n")
|
public_key, private_key = generate_keys()

print("Public Key:")
print(public_key)

print("\nPrivate Key:")
print(private_key)

message = input("\nEnter message to sign: ")

signature = sign_message(message, private_key)

print("\nDigital Signature:")
print(signature)

if verify_signature(message, signature, public_key):
    print("\nVerification: SUCCESS")
    print("Message integrity and authenticity verified.")
else:
```

```
4 user input.py - C:/Users/IT-49/AppData/Local/Programs/Python/Python314/4 user input.py (3,14,6)
File Edit Format Run Options Window Help
print("\nPrivate Key:")
print(private_key)

message = input("\nEnter message to sign: ")

signature = sign_message(message, private_key)

print("\nDigital Signature:")
print(signature)

if verify_signature(message, signature, public_key):
    print("\nVerification: SUCCESS")
    print("Message integrity and authenticity verified.")
else:
    print("\nVerification: FAILED")

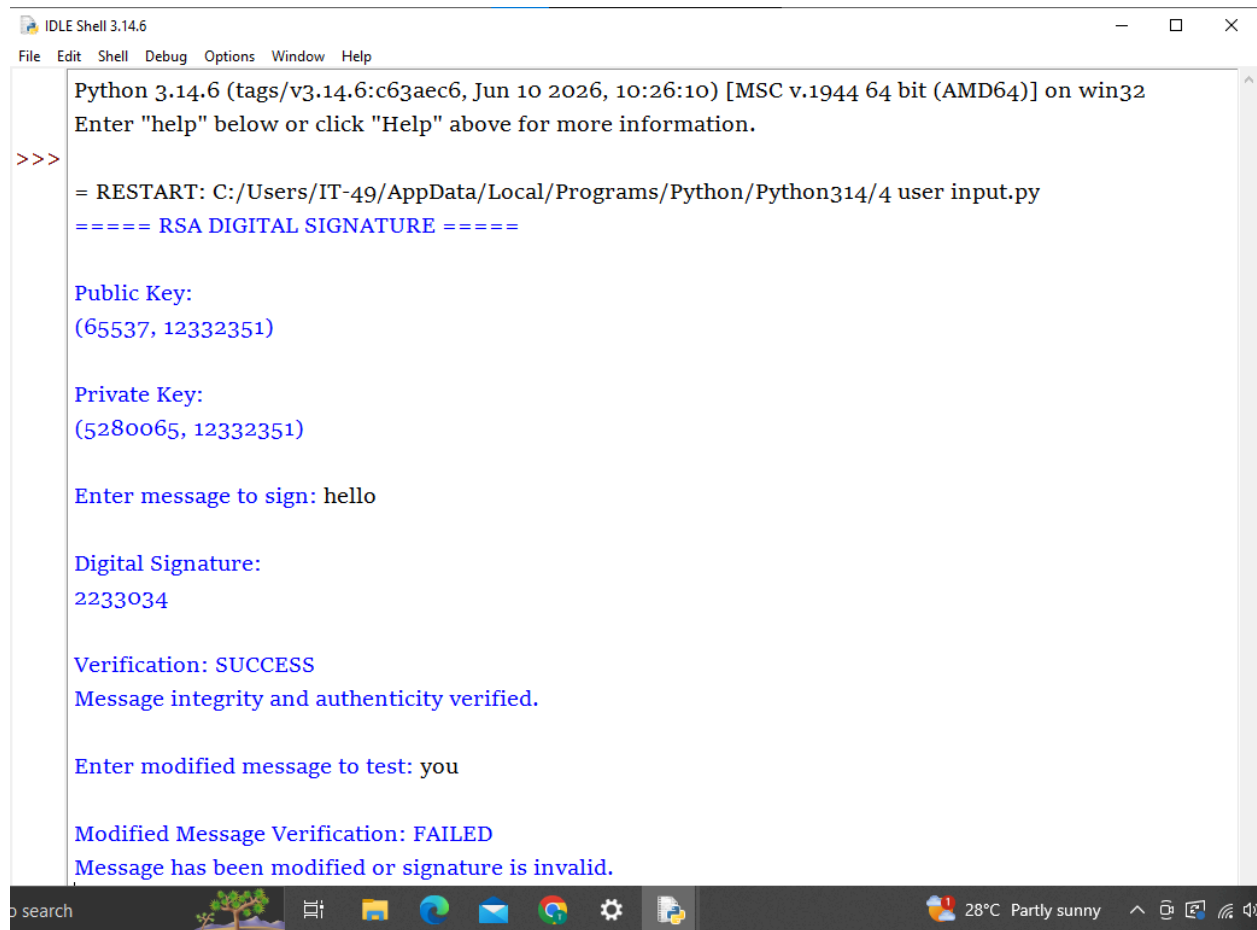
modified_message = input("\nEnter modified message to test: ")

if verify_signature(modified_message, signature, public_key):
    print("\nModified Message Verification: SUCCESS")
else:
    print("\nModified Message Verification: FAILED")
    print("Message has been modified or signature is invalid.")
```

Output ;

Name : adarsh shukla
Roll no : T117

SHETH L.U.J AND SIR M.V. COLLEGE
SUBJECT NAME: Cyber & Information Security
Practical 4 : Digital Signature



The screenshot shows a terminal window titled 'IDLE Shell 3.14.6' with a menu bar (File, Edit, Shell, Debug, Options, Window, Help). The terminal output is as follows:

```
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
= RESTART: C:/Users/IT-49/AppData/Local/Programs/Python/Python314/4 user input.py
===== RSA DIGITAL SIGNATURE =====

Public Key:
(65537, 12332351)

Private Key:
(5280065, 12332351)

Enter message to sign: hello

Digital Signature:
2233034

Verification: SUCCESS
Message integrity and authenticity verified.

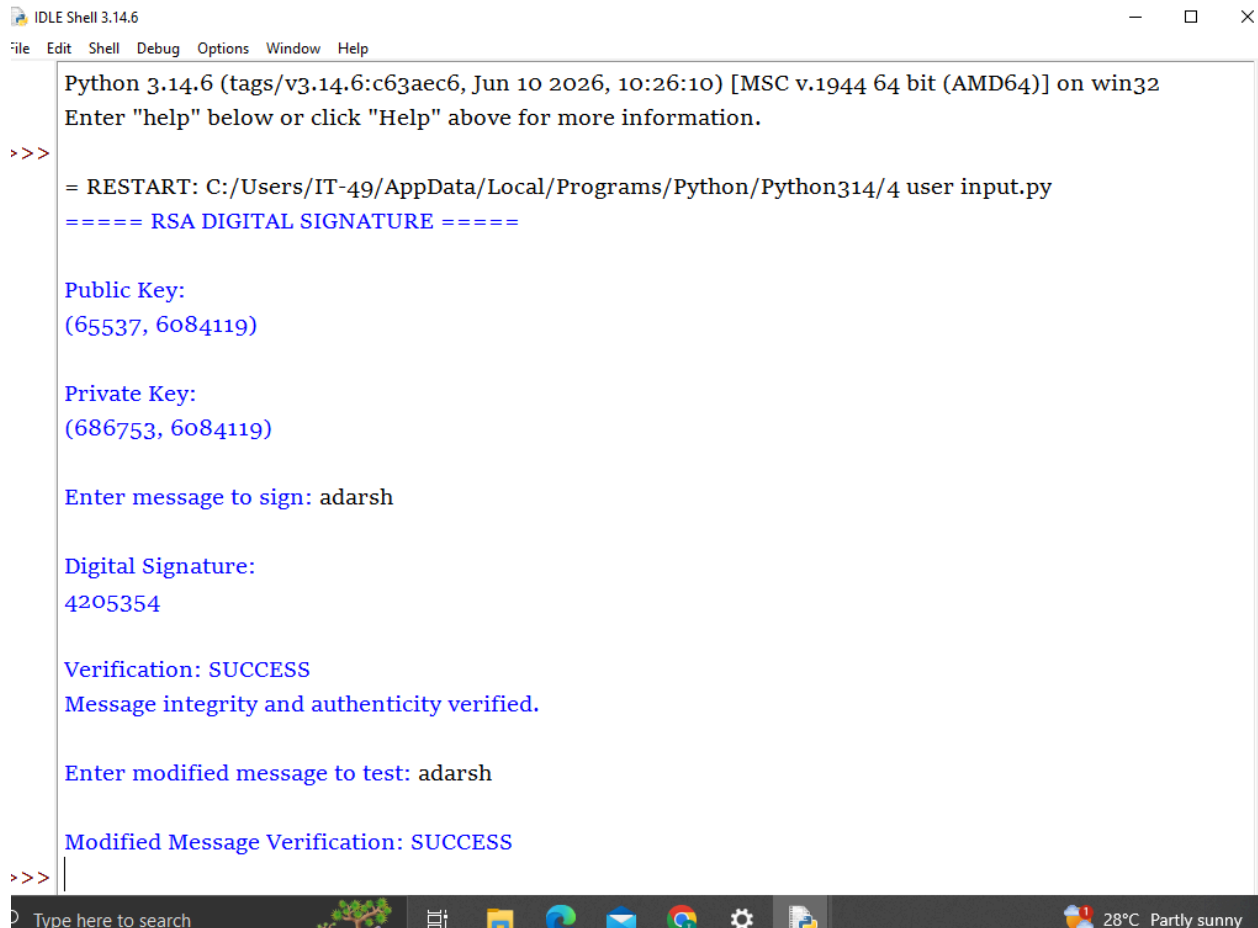
Enter modified message to test: you

Modified Message Verification: FAILED
Message has been modified or signature is invalid.
```

The Windows taskbar at the bottom shows the search bar, task view button, and several application icons (File Explorer, Edge, Mail, Chrome, Settings, and a folder icon). The system tray on the right displays the date and time (28°C Partly sunny) and icons for network, volume, and power.

Name : adarsh shukla
Roll no : T117

SHETH L.U.J AND SIR M.V. COLLEGE
SUBJECT NAME: Cyber & Information Security
Practical 4 : Digital Signature



The screenshot shows an IDLE Shell window titled "IDLE Shell 3.14.6". The window contains the following text:

```
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/IT-49/AppData/Local/Programs/Python/Python314/4 user input.py
===== RSA DIGITAL SIGNATURE =====

Public Key:
(65537, 6084119)

Private Key:
(686753, 6084119)

Enter message to sign: adarsh

Digital Signature:
4205354

Verification: SUCCESS
Message integrity and authenticity verified.

Enter modified message to test: adarsh

Modified Message Verification: SUCCESS
>>>
```

The window has a menu bar with "File", "Edit", "Shell", "Debug", "Options", "Window", and "Help". The taskbar at the bottom shows a search bar, several application icons, and a system tray with a weather widget displaying "28°C Partly sunny".

gui Implement digital signature algorithms such as RSA-based signatures, and verify the integrity and authenticity of digitally signed messages.

Name : adarsh shukla
Roll no : T117

SHETH L.U.J AND SIR M.V. COLLEGE
SUBJECT NAME: Cyber & Information Security
Practical 4 : Digital Signature

```
4gui.py - C:/Users/IT-49/AppData/Local/Programs/Python/Python314/4gui.py (3.14.6)
File Edit Format Run Options Window Help

import hashlib
import random
import math
import tkinter as tk
from tkinter import messagebox

def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(math.sqrt(n)) + 1):
        if n % i == 0:
            return False
    return True

def generate_prime():
    while True:
        n = random.randint(1000, 5000)
        if is_prime(n):
            return n

def generate_keys():
    p = generate_prime()
    q = generate_prime()
```

Ln: 7 Col: 0

```
4gui.py - C:/Users/IT-49/AppData/Local/Programs/Python/Python314/4gui.py (3.14.6)
File Edit Format Run Options Window Help

def generate_keys():
    p = generate_prime()
    q = generate_prime()

    while p == q:
        q = generate_prime()

    n = p * q
    phi = (p - 1) * (q - 1)

    e = 65537
    if math.gcd(e, phi) != 1:
        e = 3
        while math.gcd(e, phi) != 1:
            e += 2

    d = pow(e, -1, phi)

    return (e, n), (d, n)

def sign_message(message, private_key):
    d, n = private_key
    hash_value = hashlib.sha256(message.encode()).hexdigest()
    hash_number = int(hash_value, 16) % n
```

Ln: 7 Col: 0

Name : adarsh shukla
Roll no : T117

SHETH L.U.J AND SIR M.V. COLLEGE

SUBJECT NAME: Cyber & Information Security

Practical 4 : Digital Signature

```
4gui.py - C:/Users/IT-49/AppData/Local/Programs/Python/Python314/4gui.py (3.14.6)
File Edit Format Run Options Window Help

def sign_message(message, private_key):
    d, n = private_key
    hash_value = hashlib.sha256(message.encode()).hexdigest()
    hash_number = int(hash_value, 16) % n
    signature = pow(hash_number, d, n)
    return signature

def verify_signature(message, signature, public_key):
    e, n = public_key
    hash_value = hashlib.sha256(message.encode()).hexdigest()
    hash_number = int(hash_value, 16) % n
    recovered_hash = pow(signature, e, n)
    return recovered_hash == hash_number

# ===== GUI =====
public_key, private_key = generate_keys()
signature = None

def sign():
    global signature
    msg = entry_msg.get()
    if msg == "":
        messagebox.showwarning("Error", "Enter a message")

Ln: 7 Col: 0
```

```
*4gui.py - C:/Users/IT-49/AppData/Local/Programs/Python/Python314/4gui.py (3.14.6)*
File Edit Format Run Options Window Help

public_key, private_key = generate_keys()
signature = None

def sign():
    global signature
    msg = entry_msg.get()
    if msg == "":
        messagebox.showwarning("Error", "Enter a message")
    return

    signature = sign_message(msg, private_key)
    text_output.delete(1.0, tk.END)
    text_output.insert(tk.END, f"Public Key: {public_key}\n")
    text_output.insert(tk.END, f"Private Key: {private_key}\n\n")
    text_output.insert(tk.END, f"Message: {msg}\n")
    text_output.insert(tk.END, f"Signature: {signature}\n")

def verify():
    msg = entry_verify.get()
    if signature is None:
        messagebox.showwarning("Error", "Sign a message first")
    return

    result = verify_signature(msg, signature, public_key)

Ln: 56 Col: 0
```

Name : adarsh shukla
Roll no : T117

SHETH L.U.J AND SIR M.V. COLLEGE

SUBJECT NAME: Cyber & Information Security

Practical 4 : Digital Signature

```
*4gui.py - C:/Users/IT-49/AppData/Local/Programs/Python/Python314/4gui.py (3.14.6)*
File Edit Format Run Options Window Help

text_output.insert(tk.END, f"Signature: {signature}\n")

def verify():
    msg = entry_verify.get()
    if signature is None:
        messagebox.showwarning("Error", "Sign a message first")
        return

    result = verify_signature(msg, signature, public_key)

    if result:
        messagebox.showinfo("Verification", "✅ SUCCESS\nMessage is authentic")
    else:
        messagebox.showerror("Verification", "❌ FAILED\nMessage modified")

root = tk.Tk()
root.title("RSA Digital Signature")
root.geometry("500x400")

tk.Label(root, text="Enter Message:").pack()
entry_msg = tk.Entry(root, width=50)
entry_msg.pack()

tk.Button(root, text="Sign Message", command=sign).pack(pady=10)

Ln: 56 Col: 0
```

```
*4gui.py - C:/Users/IT-49/AppData/Local/Programs/Python/Python314/4gui.py (3.14.6)*
File Edit Format Run Options Window Help

    else:
        messagebox.showerror("Verification", "❌ FAILED\nMessage modified")

root = tk.Tk()
root.title("RSA Digital Signature")
root.geometry("500x400")

tk.Label(root, text="Enter Message:").pack()
entry_msg = tk.Entry(root, width=50)
entry_msg.pack()

tk.Button(root, text="Sign Message", command=sign).pack(pady=10)

tk.Label(root, text="Verify Message:").pack()
entry_verify = tk.Entry(root, width=50)
entry_verify.pack()

tk.Button(root, text="Verify Signature", command=verify).pack(pady=10)

text_output = tk.Text(root, height=10, width=60)
text_output.pack(pady=10)

root.mainloop()

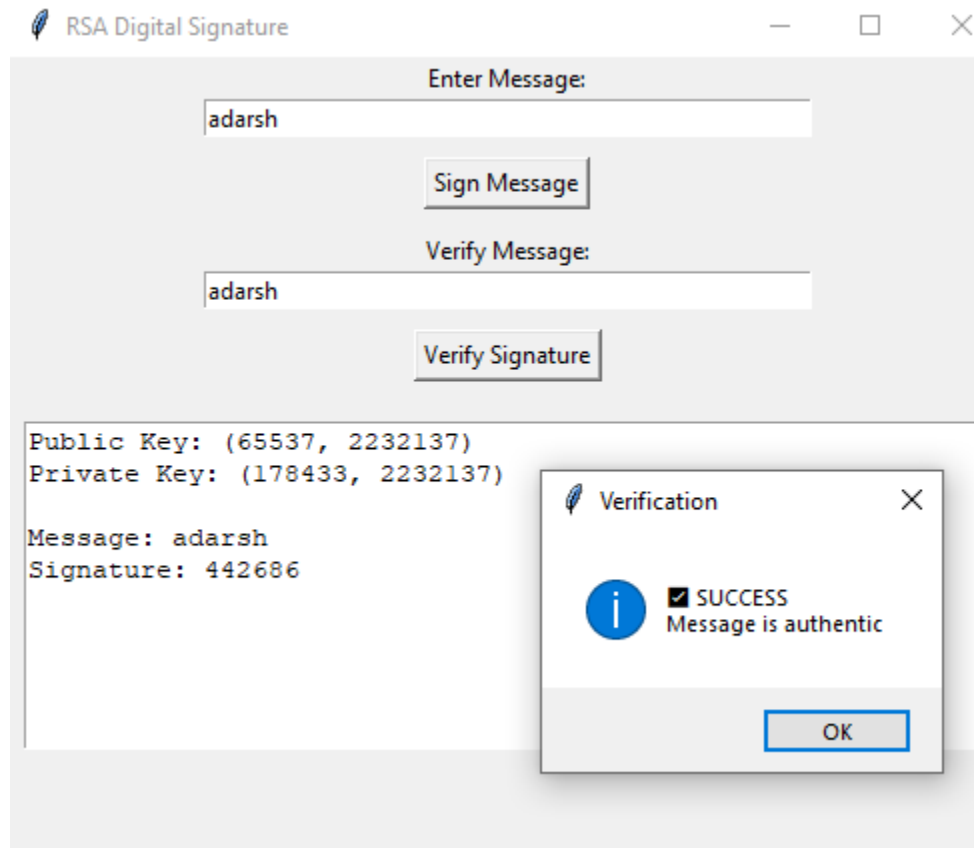
Ln: 56 Col: 0
```

Output :

Name : adarsh shukla

Roll no : T117

SHETH L.U.J AND SIR M.V. COLLEGE
SUBJECT NAME: Cyber & Information Security
Practical 4 : Digital Signature



Name : adarsh shukla
Roll no : T117

SHETH L.U.J AND SIR M.V. COLLEGE
SUBJECT NAME: Cyber & Information Security
Practical 4 : Digital Signature

